Project 3: Graph Modeling and Algorithm Report

Tarzan and Jojo Maze Problem

1. Graph Model

Graph Type: Directed, Unweighted Graph

Graph Construction:

The maze problem will be modeled as a directed graph where:

- **Vertices (Nodes):** Each vertex represents a state in the maze, defined by a position (row, column) within the grid. Only valid positions (those containing directions or Jojo's location 'J') are included as vertices. Positions marked with 'X' are not represented in the graph.
- **Edges (Directed):** A directed edge exists from vertex (r1, c1) to vertex (r2, c2) if and only if:
  1. Position (r1, c1) contains a directional indicator (N, S, E, W, NE, NW, SE, SW)
  2. Moving from (r1, c1) in the indicated direction for either 3 or 4 spaces lands exactly on position (r2, c2)
  3. Position (r2, c2) is within the maze boundaries
  4. Position (r2, c2) is not marked with 'X'

Construction Algorithm:

```pseudocode
```

```
function constructGraph(maze, rows, cols):
    graph = empty adjacency list

    for each position (r, c) in maze:
        if maze[r][c] is not 'X':
            direction = maze[r][c]
            if direction is not 'J':
                // Try both 3 and 4 space jumps
                for distance in {3, 4}:
                    (newR, newC) = calculateNewPosition(r, c, direction, distance)

                    if isValidPosition(newR, newC, rows, cols):
                        if maze[newR][newC] is not 'X':
                            addEdge(graph, (r,c), (newR,newC), direction, distance)

    return graph

function calculateNewPosition(row, col, direction, distance):
    // Calculate displacement based on direction
    deltaRow = 0, deltaCol = 0

    if direction contains 'N': deltaRow = -distance
    if direction contains 'S': deltaRow = +distance
    if direction contains 'E': deltaCol = +distance
    if direction contains 'W': deltaCol = -distance

    // For diagonal moves, distance applies to each component
    return (row + deltaRow, col + deltaCol)
```

Example: If position (3, 2) contains 'S' (south), edges would be created to:

- Position (6, 2) with move "S-3" (3 spaces south)
- Position (7, 2) with move "S-4" (4 spaces south) Only if these destination positions are valid and not marked 'X'.

2. Algorithm Selection

Algorithm: Breadth-First Search (BFS)

Justification:

- BFS is optimal for finding the shortest path in an unweighted graph
- It guarantees finding a solution if one exists

- The graph is unweighted (each move has equal cost regardless of distance)
- BFS explores paths level-by-level, ensuring we find a path with minimum number of moves

Algorithm Implementation:

```pseudocode
function findPath(graph, start, rows, cols, maze):
    queue = empty queue
    visited = empty set
    parent = empty map  // Maps position to (previous position, move taken)

    queue.enqueue(start)
    visited.add(start)
    parent[start] = null

    while queue is not empty:
        current = queue.dequeue()

        // Check if we reached Jojo
        if maze[current.row][current.col] == 'J':
            return reconstructPath(parent, current, start)

        // Explore all neighbors
        for each edge from current to neighbor in graph:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = (current, edge.move)
                queue.enqueue(neighbor)

    return "No solution exists"

function reconstructPath(parent, goal, start):
    path = empty list
    current = goal

    // Backtrack from goal to start
    while current != start:
        (previous, move) = parent[current]
        path.prepend(move)  // Add to front of list
        current = previous

    return path
```

Path Reconstruction: The parent map stores both the previous position and the move taken to reach each position. After BFS finds Jojo's location, we backtrack through the parent map from Jojo to the start position, collecting the sequence of moves. This sequence is then reversed (or prepended during collection) to produce the correct order from start to goal.

Move Representation: Each move is stored as a string in the format "DIRECTION-DISTANCE" (e.g., "S-4", "NE-3"), which directly matches the required output format.

3. Time and Space Complexity

Time Complexity: O(V + E)

- V = number of valid positions in maze (at most rows × cols)
- E = number of edges (at most 2V, since each position can have at most 2 outgoing edges)

Space Complexity: O(V)

- Graph storage: O(V + E)
- BFS auxiliary structures: O(V)

This is efficient even for the maximum maze size of 1000×1000.